INTER-PROCESS CHAT APPLICATION USING IPC

OPERATING SYSTEMS PROJECT DOCUMENTATION

Course: Operating Systems
Group: 8

Team Members and Roll Numbers:
1. Muvva Lohitha – 2520030378
2. S. Vaibhava Sree – 2520030398
3. K. Varalakshmi – 2520030578

Department of Computer Science and Engineering
KLH University
Academic Year: 2026–2027

1. Abstract

This project implements a two-way chat application between two independent processes using Linux Named Pipes (FIFOs). Process A sends a text message to Process B through one FIFO, and Process B sends a reply to Process A through a second FIFO. The application is written in C and runs on Ubuntu/Linux. It demonstrates inter-process communication (IPC), FIFO creation and access, stream-based input/output, and basic process termination using an exit message.

2. Introduction

An operating system provides mechanisms that allow independent processes to exchange information. Inter-process communication (IPC) is one such mechanism. This project demonstrates IPC using two named pipes, enabling communication in both directions between two separately executed C programs.

3. Problem Statement

Develop a two-way communication application in which independent processes exchange messages using an IPC mechanism.

4. Objectives

- To implement two-way communication between two independent processes.
- To use Linux Named Pipes (FIFOs) for inter-process communication.
- To read messages and replies through standard C input/output functions.
- To demonstrate FIFO creation, opening, reading, writing, and closing.
- To test the application by exchanging messages and terminating with the exit command.

5. Existing System and Proposed System

5.1 Existing System

Without an IPC mechanism, two independent processes do not directly exchange messages through this application's interface.

5.2 Proposed System

The proposed system uses two Linux named pipes. The first FIFO transfers messages from Process A to Process B; the second FIFO transfers replies from Process B to Process A. This arrangement provides simple, turn-based, two-way text communication.

6. Technologies Used

Technology / Tool	Purpose
C programming language	Implements Process A and Process B.
Ubuntu/Linux	Operating environment providing FIFO and POSIX facilities.
GCC compiler	Compiles the C source files into executable programs.
Linux Named Pipes (FIFOs)	Transfers messages between the independent

	processes.
GNU Nano / text editor	Creates and edits the source files.
Linux terminal	Compiles, executes, and tests the application.
Git and GitHub (if uploaded)	Version control and source-code hosting.

7. Architecture Design

The system consists of two independent processes connected by two named pipes.

- Process A (process_a.c): reads a message from the user, writes it to the A-to-B FIFO, reads Process B's reply from the B-to-A FIFO, and displays it.
- FIFO 1 — /tmp/ipc_chat_A_to_B: carries messages from Process A to Process B.
- Process B (process_b.c): reads and displays Process A's message, accepts a reply from its user, and writes the reply to the B-to-A FIFO.
- FIFO 2 — /tmp/ipc_chat_B_to_A: carries replies from Process B to Process A.

Communication flow: Process A → FIFO A-to-B → Process B → FIFO B-to-A → Process A.

Both processes use the same FIFO paths. The Linux operating system provides the named-pipe mechanism. The programs run in separate terminal windows.

8. Methodology and Algorithm

1. Start Process A and Process B in separate terminal windows.
2. Each program creates the FIFO paths if needed or checks that existing paths are FIFOs.
3. Process A opens the A-to-B FIFO for writing and the B-to-A FIFO for reading.
4. Process B opens the A-to-B FIFO for reading and the B-to-A FIFO for writing.
5. Process A reads a message from its terminal and sends it through the first FIFO.
6. Process B receives and displays the message, then reads a reply from its terminal and sends it through the second FIFO.
7. Process A receives and displays the reply; the exchange repeats.
8. When Process A sends the message 'exit', both programs leave their communication loops and close their streams.

9. Important Functions Used

Function / Macro	Purpose
mkfifo()	Creates a named pipe.
lstat()	Obtains information about a path, including an existing FIFO.
S_ISFIFO()	Checks whether the path refers to a FIFO.
open()	Opens a FIFO for reading or writing.
fdopen()	Associates an opened file descriptor with a standard I/O stream.
fgets()	Reads a line from the terminal or FIFO stream.
fprintf()	Writes a formatted message to a stream.
fflush()	Flushes buffered output.
strcmp()	Compares strings, including checking for the exit command.
fclose()	Closes a stream.

signal()	Configures handling of SIGPIPE.

10. Integration of OS Course Outcomes (COs)

The following course outcomes are directly demonstrated by the implementation. This mapping is based on the supplied project code and the provided CO syllabus.

CO-1: The OS as a Service Layer

Relevant concepts: user-space programs, Linux shell, system calls, and file descriptors. The two C programs execute in user space and use operating-system interfaces such as open() and mkfifo() to access named pipes.

CO-2: Processes and Process Control

Relevant concepts: independent processes, execution, and termination. Process A and Process B are separate programs running in separate terminal sessions. Each has its own execution flow and terminates its loop when the exit message is received or sent.

CO-3: Inter-Process Communication (IPC)

This is the primary course outcome of the project. The application uses two Named Pipes (FIFOs) for bidirectional message exchange. It also uses stream I/O, string comparison, and SIGPIPE handling.

CO-4: Memory Management

The programs use fixed-size character arrays (message and reply buffers) to store text temporarily. The implementation does not directly demonstrate virtual-memory translation, page tables, demand paging, or copy-on-write.

CO-5: File Systems and File I/O

Named FIFOs are filesystem paths under /tmp. The programs use mkfifo(), lstat(), open(), and file descriptors to create, validate, and access these IPC endpoints. fdopen(), fgets(), fprintf(), fflush(), and fclose() provide stream-based I/O.

CO-6: Concurrency and Synchronization

The two programs run as independent processes and coordinate their turn-based exchange through blocking FIFO operations. The current application does not implement pthreads, mutexes, condition variables, or semaphores.

Note: CO-3 is the main focus. CO-1, CO-2, CO-4, CO-5, and CO-6 are connected only to the extent described above; the project should not be presented as implementing every topic in those course outcomes.

11. Testing and Results

11.1 Functional Test

A successful test exchanged the message 'Hello' from Process A and the reply 'Hi' from Process B. Process A then sent 'exit', after which both processes displayed their disconnection messages.

11.2 Test Cases

Test	Action	Expected / Observed Behaviour
Normal message	A sends Hello; B replies Hi.	Message and reply are displayed

		by the receiving processes.
Exit command	A sends exit.	Both processes leave their communication loops.
Process startup	Run A and B in separate terminals.	FIFO open operations connect the processes.

12. Compilation and Execution

From the Ubuntu terminal, navigate to the project directory:

```
cd ~/IPC_Chat_Project
```

Compile both programs:

```
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L process_a.c -o process_a
gcc -Wall -Wextra -std=c11 -D_POSIX_C_SOURCE=200809L process_b.c -o process_b
```

Terminal 1:

```
./process_a
```

Terminal 2:

```
./process_b
```

Start both programs in separate terminals. Process A sends the first message; Process B enters a reply when prompted. Type exit in Process A to end the chat.

13. Limitations

- The application supports two processes and turn-based text messages only.
- It does not provide a graphical user interface, authentication, encryption, or persistent message history.
- The FIFO paths are fixed under /tmp and both programs must use the same paths.
- The programs should be stopped before removing stale FIFO paths.
- The current design is a learning prototype, not a production chat service.

14. Conclusion

The Inter-Process Chat Application demonstrates two-way communication between independent C processes using Linux Named Pipes. Two FIFOs provide separate directions for messages and replies. The project illustrates practical IPC and connects the implementation to Linux process, system-call, and file-I/O concepts.

15. Future Scope

- Add support for multiple clients or multiple chat sessions.
- Improve error handling for interrupted or disconnected processes.
- Add timestamps and message logging.
- Provide a graphical user interface.
- Add secure authentication and encryption if the project is extended beyond a classroom demonstration.

References

9. Operating Systems course syllabus (CO-1 to CO-6), supplied by the course.

10. Linux/POSIX C library documentation for mkfifo(), open(), read/write streams, and signal handling.
11. The project's process_a.c and process_b.c source files.